

Uniform Dispersion of Oblivious Mobile Robots on Alternate Nodes of an Infinite Grid

*Project report submitted
in partial fulfillment of the requirement for the degree of*

Bachelor of Engineering in Information Technology

By

Sudip Naskar (002211001131) (ITE00268082)
Subhadip Sinha (002111001154) (ITE00268083)
Subhas Chandra Shit (002211001130) (ITE00268072)

Under the guidance of

Dr. Sruti Gan Chaudhuri



Department of Information Technology,

Faculty of Engineering and Technology,

Jadavpur University, Salt Lake Campus

2025-2026

Department of Information Technology
Faculty of Engineering and Technology
Jadavpur University

BONAFIDE CERTIFICATE

This is to certify that this project report entitled “**Uniform Dispersion of Oblivious Mobile Robots on Alternate Nodes of an Infinite Grid**” submitted to **Department of Information Technology, Jadavpur University, Salt Lake Campus, Kolkata**, is a bonafide record of work done by **Sudip Naskar (Registration No: 163326 of 2022-23)**, **Subhadip Sinha (Registration No: 158871 of 2021-22)** and **Subhas Chandra Shit (Registration No: 163325 of 2022-23)** under my supervision from **25.07.2025** to **01.06.2026**.

Dr. Sruti Gan Chaudhuri
Assistant Professor(Contractual)
Department of Information Technology
Jadavpur University

Countersigned By:

Prof. Bhaskar Sardar,
Head Of the Department,
Department of Information Technology
Jadavpur University

Declaration by Students

This certifies that the report presented here is our original work. We have made sure that no part of this report has been copied or plagiarized from any other source. Wherever we used information from outside sources, we provided proper acknowledgment and citation. We take full responsibility for the authenticity of this work and accept accountability if any instance of plagiarism is found. All content, materials, and methods that are not originally developed by us have been clearly referenced. We also declare that this work is original, has not been published before.

Sudip Naskar
Roll no: 002211001131

Subhadip Sinha
Roll no: 002111001154

Subhas Chandra Shit
Roll no: 002211001130

Place: Kolkata
Date: 01.06.2026

Acknowledgments

We would like to acknowledge all those whose support and guidance made this project possible. These people have always been a tremendous support whenever we needed some motivation and guidance while working on this project.

First, we would like to thank our supervisor, Dr. Sruti Gan Chaudhuri, for providing us with the opportunity to undertake this project and for guiding us in presenting our findings. Her constant encouragement, valuable feedback, and insightful suggestions helped us improve our work at every stage. She was always available to clarify our doubts and guide us in the right direction. Without her guidance and support, this project would not have been possible.

We would also like to extend our gratitude to the University and the Department of Information Technology for providing the necessary resources and infrastructure. We are especially thankful to the technical staff of the software labs for maintaining the systems, which enabled us to work smoothly without interruptions. We also appreciate the administrative and support staff for their cooperation and assistance in handling various non-technical aspects, which contributed to the timely completion of this project.

Finally, we would like to thank our parents and family for their constant encouragement, care, and support. Their motivation has been a strong source of inspiration throughout this journey.

Department of Information Technology
Jadavpur University
Salt Lake Campus, Kolkata

Sudip Naskar
Roll no: 002211001131

Subhadip Sinha
Roll no: 002111001154

Subhas Chandra Shit
Roll no: 002211001130



JADAVPUR UNIVERSITY

Dept. of Information Technology

Vision:

To provide young undergraduate and postgraduate students a responsive research environment and quality education in Information Technology to contribute in education, industry and society at large.

Mission:

- M1:** To nurture and strengthen professional potential of undergraduate and postgraduate students to the highest level.
- M2:** To provide international standard infrastructure for quality teaching, research and development in Information Technology.
- M3:** To undertake research challenges to explore new vistas of Information and Communication Technology for sustainable development in a value-based society.
- M4:** To encourage teamwork for undertaking real life and global challenges.

Program Educational Objectives (PEOs):

Graduates should be able to:

- PEO1:** Demonstrate recognizable expertise to solve problems in the analysis, design, implementation and evaluation of smart, distributed, and secured software systems.
- PEO2:** Engage in the engineering profession globally, by contributing to the ethical, competent, and creative practice of theoretical and practical aspects of intelligent data engineering.
- PEO3:** Exhibit sustained learning capability and ability to adapt to a constantly changing field of Information Technology through professional development, and self-learning.
- PEO4:** Show leadership qualities and initiative to ethically advance professional and organizational goals through collaboration with others of diverse interdisciplinary backgrounds.

Mission - PEO matrix:

Ms/ PEOs	M1	M2	M3	M4
PEO1	3	2	2	1
PEO2	2	3	2	1
PEO3	2	2	3	1
PEO4	1	2	2	3

(3 – Strong, 2 – Moderate and 1 – Weak)

Program Specific Outcomes (PSOs):

At the end of the program a student will be able to:

- PSO1:** Apply the principles of theoretical and practical aspects of ever evolving Programming & Software Technology in solving real life problems efficiently.
- PSO2:** Develop secured software systems considering constantly changing paradigms of communication and computation of web enabled distributed Systems.
- PSO3:** Design ethical solutions of global challenges by applying intelligent data science & management techniques on suitable modern computational platforms through interdisciplinary collaboration.

Abstract

In this paper, we address a challenging problem of dispersing a set of autonomous, oblivious, homogeneous, identical mobile robots on an infinite two-dimensional grid. The robots are initially scattered on distinct nodes and eventually, through our algorithm, they distribute themselves uniformly such that they occupy alternate nodes of the grid. This uniform distribution enables the robots to cover a region of the grid with a minimal number of robots. We develop and simulate a distributed algorithm that relies on full visibility, semi-synchronous movement, and axis agreement of the robots. We rigorously analyze the movement complexity, establishing an upper bound on the number of cycles required for complete system stabilization. To address potential bottlenecks in dense initial configurations, a priority-based resolution mechanism is introduced, which reduces local congestion and prevents the formation of temporary stagnation zones. Furthermore, we evaluate the system's performance across varying initial entropy levels, demonstrating that the time-to-convergence remains robust even as the spatial distribution of the robots deviates significantly from the target pattern. We also prove that the algorithm converges in finite time and guarantees collision-free movement without entering deadlock states. We also present an illustration of the algorithm using simulation. Experimental results confirm that our approach maintains structural integrity under diverse grid geometries, offering a scalable solution for autonomous coverage tasks in resource-constrained environments. By isolating the dependency on specific grid sizes, our algorithm ensures consistent performance regardless of the total area, highlighting its versatility in dynamic field operations.

Keywords: *Mobile Robots; Distributed Algorithms; Agents; Grid Scattering; Oblivious; Semi-synchronous;*

TABLE OF CONTENTS

1	Introduction	1
1.1	System Model and Assumptions	2
1.2	Motivation and Applications.....	3
2	Earlier Works	4
2.1	Our Contribution	4
3	Problem Overview	5
4	Algorithmic Framework	6
4.1	Movement Logic	7
4.2	Behavioral and Symmetry Analysis	9
4.3	Illustrative Example and Walkthrough	10
4.4	Formal Pseudocode Specifications.....	13
	○ <i>FindYMAX</i> (n)	13
	○ <i>FindXMIN</i> (n).....	13
	○ <i>FindDimension</i> (n)	13
	○ <i>CheckXGap</i> ().....	14
	○ <i>IsUniformlyDistributed</i> ($R, rc, d, Y_{MAX}, X_{MIN}$)	14
	○ <i>FormGrid</i> (R)	15
4.5	Algorithms Implementation Mechanics	17
5	Simulation	20
5.1	Verification & Analysis.....	21
6	Correctness Proofs	22
6.1	<i>Lemma 1</i>	22
6.2	<i>Lemma 2</i>	22
6.3	<i>Theorem 1</i>	23
7	Conclusion	24
8	Future Work	25

List of Figures

1	Figure 1.....	10
2	Figure 2.....	10
3	Figure 3.....	11
4	Figure 4.....	11
5	Figure 5.....	12
6	Figure 6.....	20
7	Figure 7.....	22

CHAPTER 1

Introduction

The coordination of autonomous, homogeneous, oblivious, and identical mobile robots is a fascinating and significant area of research in distributed computing. We are particularly interested in the problem of arranging a set of n robots on an infinite grid. In our scenario, the robots are initially scattered so that no two robots occupy a single point.

Our goal is to coordinate their movement along the grid edges so that they autonomously form a specific geometric configuration: a uniform distribution in which robots are placed equidistantly apart on alternate nodes. This problem is non-trivial because the robots must achieve this pattern without central control or explicit communication through message passing.

The practical importance of this approach is most evident in large-scale coverage tasks where resources are limited. For example, in an agricultural or disinfection scenario where robots must spray a 10×10 node grid, a traditional dense deployment would require 100 robots. However, because spray or sensor ranges often overlap adjacent nodes, an alternate node distribution allows the same area to be covered using only 50 robots. This 50% reduction in hardware enables a smaller swarm to effectively guard a significantly larger geographical area. Being oblivious or stateless and having zero communication capabilities, the robots are less complex and at the same time less expensive.

In this work, we provide a deterministic solution to this problem and prove its correctness. The fundamental challenge for developing the algorithm is to handle the identical and oblivious nature of the robots and ensure the finiteness of the algorithm. We also have presented simulated results along with the theoretical proofs.

By strictly limiting the state space of each agent, we ensure that the system avoids chaotic oscillations that frequently plague decentralized swarms, instead driving the global configuration toward a stable, rigid pattern. The algorithm operates under an asynchronous scheduler, which forces the robots to be resilient against varying movement speeds and unpredictable timing, thereby guaranteeing convergence regardless of the order in which individual agents are activated.

Because the robots lack unique identifiers, symmetry breaking becomes a central requirement; our approach successfully exploits the inherent geometric constraints of the grid to resolve conflicts in decision-making among robots that perceive identical local neighbourhoods. This methodology demonstrates that complex, global order can emerge from purely local, low-memory interactions.

1.1 System Model and Assumptions

In our study, we consider a set $R = \{r_1, r_2, \dots, r_n\}$ of autonomous mobile robots deployed on an infinite grid. To ensure that our algorithm is robust and theoretically sound, we adopt the following system model:

- **Autonomous:** The robots operate independently. They execute their tasks without any intervention from a central control unit or external server.
- **Anonymous and Homogeneous:** The robots are homogeneous entities. They lack unique identifiers (IDs) and are indistinguishable from one another, which ensures our algorithm does not rely on specific robot identities.
- **Oblivious:** The robots are oblivious, meaning they possess no memory of past actions or states. Their decisions in any given cycle depend solely on their current observation, making the system self-stabilizing.
- **No Communication:** The robots cannot exchange explicit messages (e.g., via wireless signals). Instead, each robot is equipped with a camera to take 360-degree snapshots, allowing them to locate the positions of others.
- **Point Robots:** The robots are modeled as points on the grid. By abstracting away physical dimensions, the model allows us to focus strictly on node-based spatial coordination to remove the complexities of collision volume, enabling clear mathematical proofs for movement and convergence.
- **Unlimited Visibility:** The robots have unlimited visibility, allowing them to observe the entire grid and the positions of all other robots to make informed decisions.
- **Semi-Synchronous Scheduler (SSYNC):** The robots agree on a Semi-Synchronous (SSYNC) timing model. Time is divided into discrete Look-Compute-Move cycles. In our model, the scheduler activates a random subset of robots to execute a cycle synchronously. Crucially, the scheduler guarantees that a robot never captures a snapshot of another robot while it is in motion.
- **Coordinate System:** In contrast to fully local models, we assume the existence of a common global coordinate system. All robots agree on a shared global origin (0,0) as well as the direction and orientation of the X and Y axes.

1.2 Motivation and Applications

The deployment of multi-agent robotic systems has emerged as a cornerstone of modern distributed computing, driven by the need to execute large-scale operational tasks that are either too hazardous, vast, or complex for a single centralized entity. Among these, spatial coordination and uniform coverage on geometric grids represent a fundamental class of problems. Traditional multi-agent frameworks often rely heavily on expensive, high-spec hardware configurations where agents possess unique identifiers, substantial internal memory to track system history, and continuous wireless communication channels to coordinate movements. However, in real-world deployment scenarios such as wide area agricultural monitoring, automated environmental sampling, or structural disinfection maintaining these assumptions introduces significant vulnerabilities, including channel congestion, high power consumption, susceptibility to cyber-interference, and high manufacturing costs.

To address these limitations, our research focuses on a minimalist hardware model utilizing autonomous, homogeneous, oblivious, and non-communicating mobile robots. Eliminating active communication transceivers significantly reduces the energy footprint and per unit production cost, making large scale swarms economically viable. Furthermore, removing the reliance on internal memory (oblivious agents) yields an inherently robust, self-stabilizing system capable of recovering from transient sensor errors or physical displacements without requiring a system reset.

The practical utility of this minimalist swarm is most profoundly realized through strategic spatial placement, specifically an alternate-node distribution on a discrete grid. In operations like precision agriculture or chemical spraying, sensor and payload ranges naturally overlap adjacent grid coordinates. A dense deployment where every node is occupied results in 100% hardware utilization to cover a given area. Conversely, by precisely coordinating a zero-communication swarm to autonomously organize onto alternate nodes, the exact same geographical area can be monitored or treated using only 50% of the hardware resources. This 50% hardware reduction vastly extends the operational scale of a resource-constrained swarm, transforming localized multi-agent coordination into a highly scalable, economically optimized solution for extensive coverage operations in challenging, communication denied environments.

CHAPTER 2

Earlier Works

The robot coordination problem has been extensively studied by many researchers under various models. Suzuki and Yamashita [1] established the foundational model for distributed anonymous mobile robots, which inspired our approach to geometric pattern formation. Their work introduced the fundamental Look-Compute-Move cycle that serves as the backbone for almost all subsequent research in swarm robotics. Since then, the field has evolved significantly, shifting from basic pattern generation toward more complex challenges like environment exploration and self-organization in resource constrained settings.

Several researchers have focused on the Dispersion problem. For instance, Kshemkalyani et al. addressed dispersion on arbitrary graphs [2], [3] and later extended their work to grid structures [4]. Similarly, Hsiang et al. [5], [6] and Cohen and Peleg [7] provided algorithms for dispersing robot swarms in unknown environments. These studies have primarily emphasized the necessity of efficient coverage, aiming to minimize the total time required for a group of scattered agents to attain a state of maximum distribution.

2.1 Our Contribution

Our work differs from these studies. While Barriere et al. [8] and Bose et al. [9] discussed uniform scattering and arbitrary pattern formation, we specifically tackle the constraint of scattering robots onto alternate nodes. We build upon the optimizations proposed by Poudel and Sharma [10], but we tailor our solution to the semi-synchronous model with axis agreement, ensuring a robust and deadlock-free execution. Unlike previous approaches that often necessitate extensive communication or high memory capacity to resolve ambiguities, our protocol achieves the target distribution through purely local, stateless operations. This advancement directly addresses the scalability bottleneck in large-scale deployments, where reducing the computational overhead per robot is critical for maintaining long term operational viability in harsh or inaccessible terrains.

The integration of axis agreement further distinguishes our algorithm, as it simplifies the coordination logic by providing a common reference frame for movement. By utilizing this shared global orientation, our agents can compute local target configurations without the need for complex, relative coordinate transformations, which often increase the likelihood of computational errors. Consequently, our proposed deterministic algorithm not only guarantees the final desired configuration but also ensures that the transition path is optimal in terms of step counts, providing a significant improvement over non-deterministic or heuristic based methods currently used in the industry.

CHAPTER 3

Problem Overview

Our objective is to uniformly distribute the robots along a grid such that they are positioned on alternate nodes. Initially, all robots are on distinct nodes, and our protocol ensures that this separation is maintained throughout the entire transition process, preventing any collisions or node-sharing during movement.

In our model, the robots execute a standard Look-Compute-Move cycle:

- **Look:** Robots collect the positions of other robots within their sensing range, capturing a full snapshot of the swarm's global configuration without distorting the actual coordinates.
- **Compute:** In the compute state, the robots execute our deterministic algorithm for computing destinations. This algorithm is identical for all robots, ensuring consistent behaviour across the swarm without the need for unique identification.
- **Move:** The robots move to the computed destinations in this state. The scheduler ensures that no robot initiates a move if another agent is currently in transit, thereby preserving the integrity of the observational data for the next Look cycle.

In the Compute phase, each robot r_i executes the following steps:

1. **Compute Bounds:** Each robot determines the North-most bound (Y_{MAX}) and West-most bound (X_{MIN}) of the entire swarm distribution, establishing a common reference frame that allows every agent to align its internal calculation with the collective boundary of the system.
2. **Execute FORMGRID:** This routine calculates the specific target coordinate for the robot, effectively mapping its current position to an unoccupied node that satisfies the alternate-node spacing requirement. The routine iteratively evaluates path availability to ensure movement strictly follows grid edges.
3. **Validate Configuration:** Finally, the robot Validate the final configuration of the robots by executing the ISUNIFORMLYDISTRIBUTED routine. This validation step serves as a self-stabilizing safeguard, confirming that the robots are correctly positioned and that the global configuration meets the target geometric constraints.

This three-stage procedural cycle is inherently robust, as it transforms a disordered initial state into a highly structured grid topology. By embedding the validation logic directly into the Compute phase, we eliminate the need for an external observer or central controller to verify the final swarm structure, thereby allowing the system to achieve closure autonomously and efficiently.

CHAPTER 4

Algorithmic Framework

To ensure unambiguous replication of the coordination framework, Table I consolidates the mathematical notations, coordinate systems, and logical parameters utilized throughout the algorithmic formulations.

TABLE I: System Parameter Descriptions

Symbol	Mathematical Representation	Definition / Operational Role
R	$\{r_1, r_2, \dots, r_n\}$	The finite set of autonomous, oblivious, and anonymous mobile robotic agents.
$T_{x,y}$	$\mathbb{Z} \times \mathbb{Z}$	A discrete coordinate node situated within the infinite two-dimensional grid infrastructure.
Y_{MAX}	$\max_{r_i \in R}(y_i)$	The dynamic global northernmost boundary coordinate calculated during the Look phase.
X_{MIN}	$\min_{r_i \in R}(x_i)$	The dynamic global westernmost boundary coordinate calculated during the Look phase.
r_c	$\lceil \sqrt{n} \rceil$	The maximum target capacity constraint dictating row-wise and column-wise agent limits.
d	$(r_c \times 2) - 1$	The absolute distribution depth defining the vertical matrix bounds of $grid_{final}$.
j	$Y_{MAX} - y + 1$	The inverted logical row index used for top-down spatial positioning.
$\Psi_1 \dots \Psi_5$	Case Ψ_k	The 5 localized state configurations governing agent movement.

Let n be the number of robots in R . A grid node is denoted as $T_{x,y}$. Let $grid_{final}$ be the target uniform distribution where robots occupy alternate nodes. Let r_c be the maximum number of robots in a row or column of $grid_{final}$ and let d be the maximum number of rows or columns. Since alternate rows and columns will be empty, the dimensions are determined as:

$$r_c = \lceil \sqrt{n} \rceil, \quad d = (r_c \times 2) - 1$$

To coordinate movement, each robot r_i at position (x, y) calculates a logical row index:

$$j = Y_{MAX} - y + 1$$

Under this mapping, the North-most row corresponds to $j = 1$, which allows the robots to determine their specific configuration and target destination during the Compute phase. To execute this mapping, the system utilizes a suite of algorithms to guide robots from initial chaos to a structured grid. Algorithms 1 and 2 establish workspace boundaries, while Algorithm 5 calculates the grid's scale. Subsequently, Algorithm 6 monitors horizontal spacing to prevent overlaps, leading to Algorithm 4, which governs the robot movement process. Finally, Algorithm 3 validates the formation. Together, these steps transform raw coordinates into a stable and collision free formation.

Furthermore, this framework completely bypasses the need for persistent localized memory or state tracking logs. If a robot is physically displaced or suffers a transient sensor failure midway through execution, the next Look-Compute-Move cycle treats the corrupted position as a fresh initial configuration. The self-stabilizing nature of the equations ensures that the swarm will dynamically adjust its boundaries and continue converging toward $grid_{final}$ without entering infinite error-handling loops.

4.1 Movement Logic

Robots compute a path from their source node $T_{x,y}$ to a destination. In case of a tie (multiple possible moves), the priority is: West > East > South > North. Depending on the position of robot r_i , there are 5 configurations, denoted as Case Ψ_1 to Case Ψ_5 .

1. **Case Ψ_1 (Even Row within Bounds):** Robot r_i is in an even row j such that $j \leq d$. In this case, r_i tries to move Northwards to an odd row.
 - r_i checks if $T_{x,y+1}$ is vacant. If so, it moves there.
 - If $T_{x,y+1}$ is not empty, r_i moves East to $T_{x+1,y}$ and then checks if the immediate North node is vacant.
 - If $T_{x+1,y}$ is also occupied, r_i waits. This continues until r_i reaches the odd row northwards.

2. **Case Ψ_2 (Beyond Distribution Depth):** Robot r_i is in a row j where $j > d$. The robot attempts to move Northwards to reach the d –th row of $grid_{final}$.
 - r_i checks if $T_{x,y+1}$ is vacant and moves there.
 - If occupied, it moves East to $T_{x+1,y}$ and then attempts to move North.
 - This continues until r_i reaches row d .
3. **Case Ψ_3 (Alignment in Odd Rows):** All robots are in alternate (odd) rows within the first d rows. However, r_i is not on an alternate node or at the West bound.
 - **Move West:** If any robot to the West of r_i has a vacant target node, r_i tries to move West. It checks if $T_{x-1,y}$ and $T_{x-2,y}$ are vacant. If so, it moves to $T_{x-1,y}$.
 - **Move East:** If all robots to the West are blocked (no vacant target nodes), r_i moves East to position itself on an alternate node. It checks if $T_{x+1,y}$ and $T_{x+2,y}$ are vacant; if so, it moves to $T_{x+1,y}$.
4. **Case Ψ_4 (Filling Northward Rows):** All robots are placed in alternate nodes of alternate rows. Robot r_i is in row j . If r_i is among the first rc robots in its row, and the Northward alternate row node $T_{x,y+2}$ is empty, r_i moves to $T_{x,y+2}$.
5. **Case Ψ_5 (Handling Excess Robots):** Robot r_i is in a row that has $\geq rc$ robots (excess robots).
 - **If r_i is in the North-most row:** It tries to move to a Southward odd row. It checks if $T_{x,y-2}$ is vacant. If yes, it moves there. Once in the new row, it moves West to align itself.
 - **If r_i is not in the North-most row:** The robot checks if odd rows above it have fewer than rc robots. If yes, and $T_{x,y+2}$ is vacant, it moves North. Otherwise, if rows above are full, it checks the Southward node $T_{x,y-2}$ and moves there if vacant.

4.2 Behavioral and Symmetry Analysis

The execution priority scheme (West > East > South > North) acts as the primary tool for symmetry breaking across the system. Because the oblivious mobile units share an identical programming logic and lack individual identifiers, a completely symmetric layout would inherently trigger synchronized, mirroring actions under a standard scheduler. By enforcing an asymmetric, directional hierarchy, the protocol guarantees that adjacent robots encountering identical local environmental layouts will compute distinct, non-conflicting operational targets. This strict arrangement ensures that multi-agent collisions are mathematically impossible, even when a high density of agents is compressed within a small grid section.

Additionally, the transition between these five distinct configuration cases creates a predictable, monotonic progression toward the target distribution. The algorithmic sequence forces a top down, left to right settling effect across the grid matrix. Robots trapped in the blocked regions of *Case Ψ_1* and *Case Ψ_2* systematically clear out the even numbered rows, filtering upward until they trigger the horizontal sorting rules of *Case Ψ_3* . As rows reach their maximum structural capacity (r_c), excess agents are pushed away into available vacant zones through *Case Ψ_4* and *Case Ψ_5* . This sequential progression ensures that local movements always contribute to a global reduction in system entropy, culminating in a stable, permanent termination state.

This directional asymmetry also plays a vital role in preventing permanent execution loops, a common failure mode in decentralized multi-agent coordination where robots cycle endlessly between identical states. In a traditional symmetric framework, two opposing agents might continuously step toward the same vacant node, notice the conflict, step back simultaneously, and repeat the loop indefinitely. By forcing the swarm to strictly prioritize the West and East channels over vertical paths, our design guarantees that one agent's trajectory will always take absolute mathematical precedence over its neighbor's pathing choice. This deterministic override forces a quick, clean break in any potential spatial gridlock, ensuring that local hesitations are resolved within a single execution cycle and keeping the entire swarm moving forward toward a stable configuration.

4.3 Illustrative Example and Walkthrough

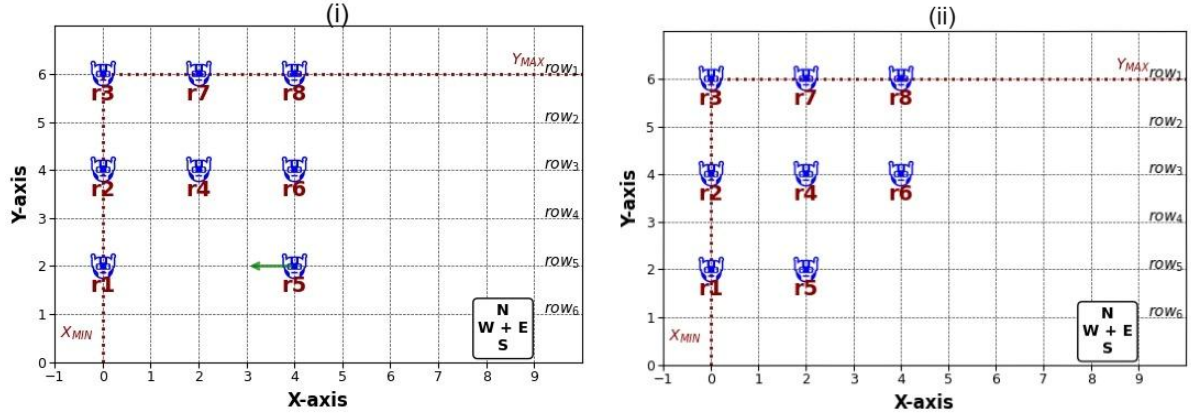


Figure 1: (i) The robots on the even rows move northwards to the odd rows of $grid_{final}$. (Case Ψ_1) and since r_1 is beyond the first d (i.e. $(3 \times 2) - 1 = 5$) rows from Y_{MAX} , so it moves northwards to the 5th row. (Case Ψ_2)

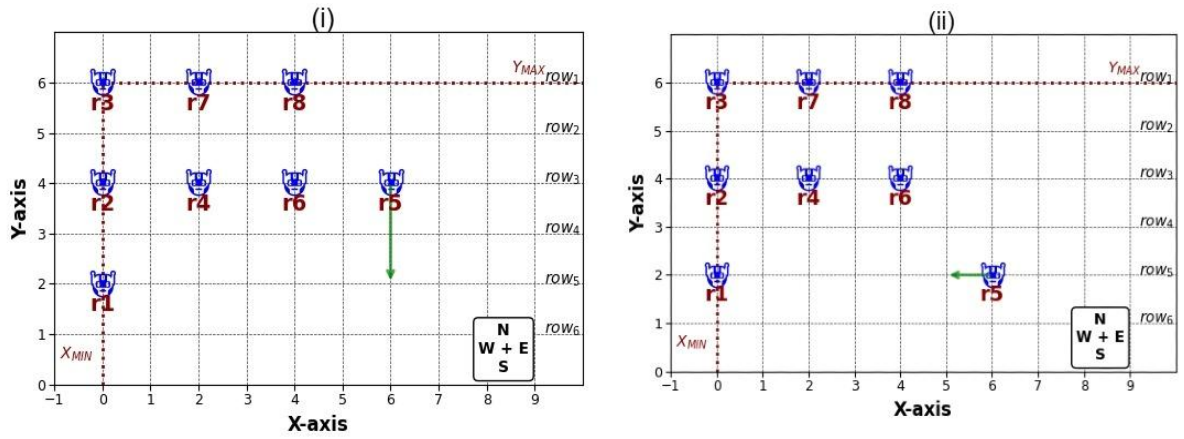


Figure 2: (i) and (ii) The robots (r_1, r_3, r_7) move to their immediate west node on the same row when the two consecutive west nodes are vacant. Robots (r_5, r_8) try to move eastwards only when there is no vacant target node to move west. (Case Ψ_3). The robots move further to place themselves on alternate nodes of the same row. (Case Ψ_3)

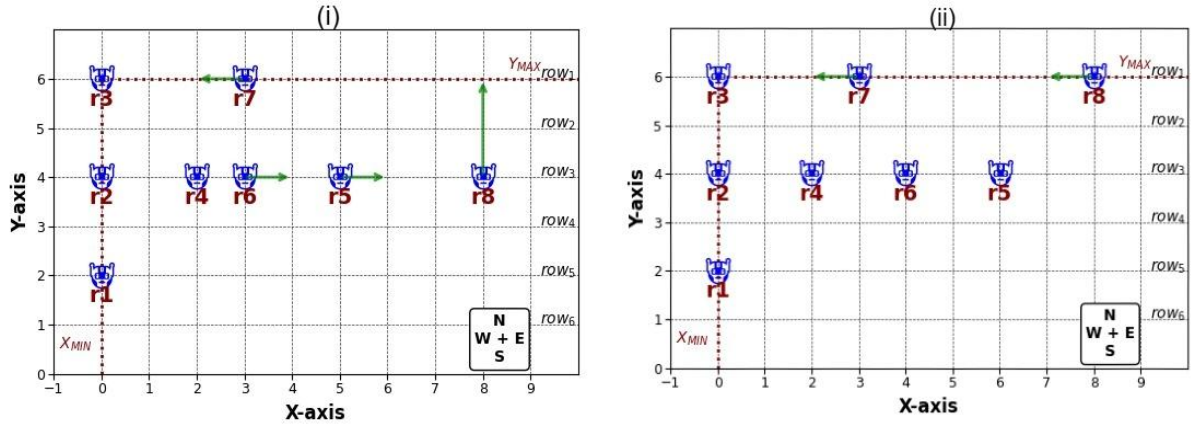


Figure 3: (i) The robot, $r_8 \notin$ first rc robots of row_3 , also moves northwards to the alternate row, row_1 , as it finds a vacant node. (Case Ψ_5) (ii) Robot r_7 and r_8 in row_1 again continue to hop west, since it again finds consecutive vacant nodes on its west. (Case Ψ_3)

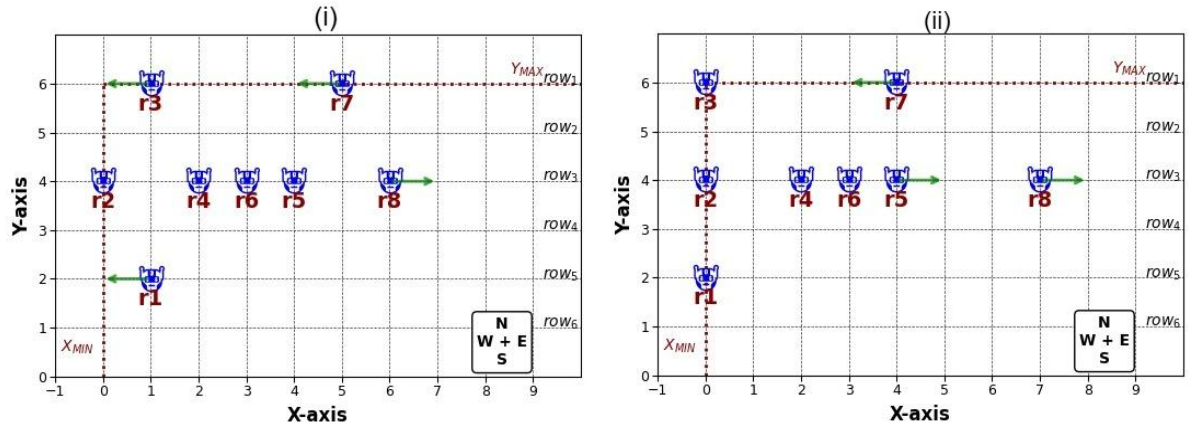


Figure 4: (i) Robot r_5 in row_3 has rc robots on its west in row_3 and the above odd row row_1 has rc robots. (Case Ψ_5). So, r_5 moves southwards to alternate row, row_5 (ii) Then robot r_5 moves westwards to place itself on an alternate node of the grid distribution.

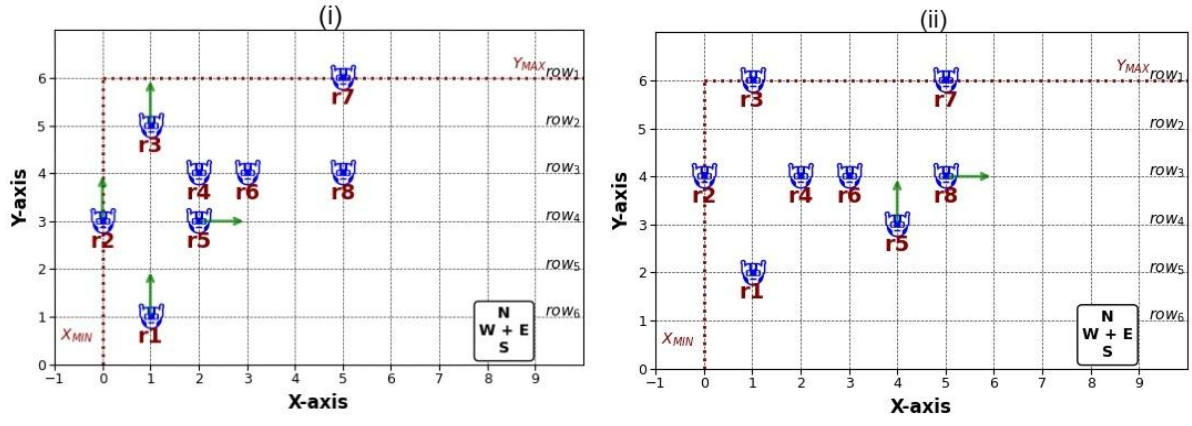


Figure 5: (i) Then Robot r_5 again move westwards to place itself on an alternate node of the grid distribution. (ii) A uniform distribution along the grid ($grid_{final}$.) is formed, with robots on alternate nodes of this grid distribution.

4.4 Formal Pseudocode Specifications

Algorithm 1: FindYMAX(n)

Input: n

Output: The north-most bound of the distribution, Y_{MAX}

Let Y_{MAX} be the Y-axis value of the north-most robot;

Initially let, $Y_{MAX} \leftarrow$ Y-axis value of r_1 ;

Let c be the robot counter to compare all robots and $c \leftarrow 2$;

while $c \leq n$ **do**

if Y-axis value of $r_c > Y_{MAX}$ **then**
 $Y_{MAX} \leftarrow$ Y-axis value of r_c ;
 $c \leftarrow c + 1$;

return Y_{MAX}

Algorithm 2: FindXMIN(n)

Input: n

Output: The west-most bound of the distribution, X_{MIN}

Let X_{MIN} be the X-axis value of the west-most robot;

Initially let, $X_{MIN} \leftarrow$ X-axis value of r_1 ;

Let c be the robot counter to compare all robots and $c \leftarrow 2$;

while $c \leq n$ **do**

if X-axis value of $r_c < X_{MIN}$ **then**
 $X_{MIN} \leftarrow$ X-axis value of r_c ;
 $c \leftarrow c + 1$;

return X_{MIN}

Algorithm 3: FindDimension(n)

Input: n

Output: Maximum grid dimensions of the uniform distribution of robots

Let $grid_{final}$ denote the uniform robot distribution with robots placed on alternate nodes;

r_c be the maximum number of robots in a row or column of $grid_{final}$;

d be the maximum number of rows or columns of $grid_{final}$;

$r_c \leftarrow \lceil \sqrt{n} \rceil$;

$d \leftarrow (r_c \times 2) - 1$;

return r_c, d

Algorithm 4: CheckXGap()

Input: Set of robots with coordinates (x, y) **Output:** *True* if row-wise spacing between all robots is 2, else return *False*Initialize an empty map *rows*;Here rows be a mapping of *Y*-axis values to lists of *X*-axis values;**foreach** list *xs* in *rows.values()* **do** Sort *xs* in ascending order; **for** $i \leftarrow 1$ **to** $|xs| - 1$ **do** **if** $xs[i] - xs[i - 1] \neq 2$ **then** **return** *False*;**return** *True*;

Algorithm 5: IsUniformlyDistributed ($R, rc, d, Y_{MAX}, X_{MIN}$)

Input: Set of Robots R , dimensions d, rc , bounds Y_{MAX}, X_{MIN} **Output:** Boolean *True* if distribution is complete, else return *False*Group robots by row index j , where $j \leftarrow Y_{MAX} - Y_{coord} + 1$;**if** maximum row index $> d$ **then** **return** *False*;**foreach** row j and its upper neighbor j_{upper} **do** **if** $count(j_{upper}) > count(j)$ **then** **return** *False*; **if** $count(j_{upper}) == count(j)$ **and** $count(j) < rc$ **then** **return** *False*;**foreach** row j **do** **if** j is even ($j \bmod 2 = 0$) **then** **return** *False*; **if** j is odd **and** number of robots in row $j > rc$ **then** **return** *False*; **if** row is not empty **and** min *X*-value $\neq X_{MIN}$ **then** **return** *False*; **if** robots in row j are not spaced by 2 units **then** **return** *False*;**return** *True*;

Algorithm 6: FormGrid(R)

Input: Set of n robots R

Output: Robots move to form a uniform grid distribution

Calculate $rc, d \leftarrow \text{FindDimension}(n)$;

rc be the maximum number of robots in a row or column of $grid_{final}$;

d be the maximum number of rows or columns of $grid_{final}$;

Calculate bounds $Y_{MAX} \leftarrow \text{FindYMAX}(n)$; $X_{MIN} \leftarrow \text{FindXMIN}(n)$;

repeat

foreach robot $r_i \in R$ **do**

 Let j be the logical row index of r_i ($j \leftarrow Y_{MAX} - r_i.y + 1$) ;

 Priority of movement: West > East > South > North;

 // Case $\Psi 1$ (Robot in even row within bounds)

if $j \leq d$ and $j \pmod{2} = 0$ **then**

if $T_{x,y+1}$ is empty **then**

r_i moves to $T_{x,y+1}$; $j \leftarrow j - 1$;

else if $T_{x+1,y}$ is empty **then**

r_i moves to $T_{x+1,y}$; $j \leftarrow j - 1$;

else

r_i waits;

 // Case $\Psi 2$ (Robot beyond distribution depth)

else if $j > d$ **then**

if $T_{x,y+1}$ is empty **then**

r_i moves to $T_{x,y+1}$; $j \leftarrow j - 1$;

else if $T_{x+1,y}$ is empty **then**

r_i moves to $T_{x+1,y}$; $j \leftarrow j - 1$;

else

r_i waits;

 // Case $\Psi 3$ (Odd row alignment)

if $j \pmod{2} = 1$ **and** other robots on odd logical indexed rows

and r_i not aligned correctly **then**

if r_i is near X_{MIN} **and** $T_{x-1,y}$ is empty **then**

r_i moves to $T_{x-1,y}$;

else if $T_{x-1,y}$ **and** $T_{x-2,y}$ is empty **and** West move capable **then**

r_i moves to $T_{x-1,y}$;

else if $T_{x+1,y}$ **and** $T_{x+2,y}$ is empty **then**

r_i moves to $T_{x+1,y}$;

else

r_i waits;

repeat
foreach robot $r_i \in R$ **do**

 // Case Ψ_4 (Filling North rows)

if $j \pmod{2} = 1$ **and** *other robots on odd logical indexed rows* **and**
CheckXGap() **then**
if $r_i \in$ *first rc robots* **and** $T_{x,y+2}$ *is empty* **then**

 └ r_i moves to $T_{x,y+2}$;

 // Case Ψ_5 (Handling excess robots)

if $j \pmod{2} = 1$ **and** *CheckXGap()* **and** *no of robots in west $\geq rc$* **then**
if $j = 1$ **then**
if $T_{x,y-2}$ *is empty* **then**

 └ r_i moves to $T_{x,y-2}$;

if *no of robots in west $< rc$* **and** $T_{x-1,y}$ **and** $T_{x-2,y}$ *is empty* **then**

 └ r_i moves to $T_{x-1,y}$;

else
if *other robots above odd rows $< rc$* **and** $T_{x,y+2}$ *is empty* **then**

 └ r_i moves to $T_{x,y+2}$;

else $T_{x,y-2}$ *is empty* **then**

 └ r_i moves to $T_{x,y-2}$;

if *no of robots in west $< rc$* **and** $T_{x-1,y}$ **and** $T_{x-2,y}$ *is empty* **then**

 └ r_i moves to $T_{x-1,y}$;

until *IsUniformlyDistributed* ($R, rc, d, Y_{MAX}, X_{MIN}$) **return** **True** ;

4.5 Algorithms Implementation Mechanics

To understand how the system executes these steps without active communication, we must look at the specific code structures that run during the **Compute** phase of the Look-Compute-Move cycle.

Boundary Tracking Logic

The system establishes its reference frames using two tight linear loops: *FindYMAX* and *FindXMIN*. These functions operate as deterministic search routines with a time complexity of $O(n)$.

- ***FindYMAX*** works by assuming the first detected robot (r_1) defines the initial northern boundary. A counter variable c initialized to 2 increments through the swarm array, evaluating the Y-coordinate of each subsequent robot r_c . If the condition $Y(r_c) > Y_{MAX}$ evaluates to true, the upper boundary updates instantly.

```
def find_YMAX(self): # Algorithm 1
    return max(r.y for r in self.robots)
```

- ***FindXMIN*** runs a parallel structural logic, evaluating whether $X(r_c) < X_{MIN}$ to lock in the westernmost edge. Because these bounds are calculated at the beginning of every single cycle, any physical movement by the boundary defining robots shifts the reference frame dynamically for the next iteration, making the algorithm entirely self-correcting.

```
def find_XMIN(self): # Algorithm 2
    return min(r.x for r in self.robots)
```

Structural Dimension Sizing

Once the edge bounds are established, the system calls *FindDimension* to calculate the structural parameters of the target matrix. The routine uses a ceiling square root function ($\lceil \sqrt{n} \rceil$) to determine r_c , which dictates the maximum capacity of robots allowed per row or column. To translate this into an interleaved grid footprint where alternate paths are kept vacant, the routine outputs the absolute distribution depth d via the formula $d = (r_c \times 2) - 1$. This mathematical step ensures that the scale of the destination grid is perfectly proportional to the swarm population n , completely preventing grid saturation.

```
def find_dimension(self): # Algorithm 3
    n = len(self.robots)
    rc = math.ceil(math.sqrt(n))          # max robots in row
    d = (rc * 2) - 1                      # max rows
    return rc, d
```

Gap Verification and Termination

To ensure stability, the swarm continuously evaluates spatial balance using row sorting and counter checks:

- **CheckXGap** acts as a local alignment filter. It extracts the X-axis coordinate maps for each active row array, sorts them in ascending order, and uses an iterative loop to check the step differences between adjacent elements ($xs[i] - xs[i - 1]$). If any step size deviates from exactly 2, the function breaks and returns False, keeping the localized alignment phase active.

```
def check_x_gap(self): # Algorithm 4
    rows = {}
    for r in self.robots:
        rows.setdefault(r.y, []).append(r.x)

    for xs in rows.values():
        xs.sort()
        for i in range(1, len(xs)):
            if xs[i] - xs[i-1] != 2:
                return False
    return True
```

- **IsUniformlyDistributed** functions as the master terminal gate. It tracks agent distribution by comparing some constraints. Execution only stops when these verification modules return a unanimous True state across all active agents. Specifically, the function groups the swarm by logical row indexes and systematically processes the configuration through the following check gates:
 - **Dimensional Boundaries and Monotonic Row Density:** The algorithm first ensures no robot exceeds the calculated target grid depth boundary. It then checks that the distribution follows a strict vertical tapering rule by verifying that no upper row (j_{upper}) contains a higher cardinality of robots than the row immediately below it ($len(rows[upper]) > len(rows[lower])$). Additionally, it flags an incomplete configuration as invalid if adjacent rows share identical counts but fall short of the maximum row capacity r_c .
 - **Parity and Alignment Constraints:** The function strictly enforces the interleaved layout criteria by verifying that no robots reside on even logical rows ($y \bmod 2 = 0$) and that odd rows do not burst past the r_c capacity threshold. It simultaneously checks the spatial alignment tracking vectors, verifying that the westernmost robot in every occupied row lines up exactly with the absolute global boundary X_{MIN} .

```

def is_Uniformly_Distributed(self, rc, YMAX, XMIN, dimension): # Algorithm 5
    # Group robots by row (y-coordinate)
    rows = {}
    for r in self.robots:
        rows.setdefault(YMAX - r.y + 1, []).append(r.x)
    # Sort y descending (top → bottom)
    sorted_rows = sorted(rows.keys(), reverse=True)
    if max(sorted_rows) > dimension: return False

    # Checking that there is any upper row has less robot than the lower row (base
    # on actual y-axis of hte robot not Ymax)
    for i in range(1, len(sorted_rows)):
        upper = sorted_rows[i - 1] # higher (top) row
        lower = sorted_rows[i]     # lower (bottom) row
        if len(rows[upper]) > len(rows[lower]):
            return False

    # Checking that there is any upper row has same robot than the lower row (base
    # on actual y-axis of hte robot not Ymax)
    for i in range(1, len(sorted_rows)):
        upper = sorted_rows[i - 1] # higher (top) row
        lower = sorted_rows[i]     # lower (bottom) row
        if len(rows[upper]) == len(rows[lower]) < rc:
            return False

    # Other Flags/Checker
    for y in sorted_rows:
        if y % 2 == 0:
            return False
        if y % 2 == 1 and len(rows[y]) > rc:
            return False

        if rows[y] and min(rows[y]) != XMIN:
            return False

        if y == dimension and max(rows[y]) == dimension - 1:
            j = YMAX - dimension + 1
            target_robot = next((r for r in self.robots if r.x == dimension
-1 and r.y == j), None)
            if target_robot and not self.check_x_gap_at_r(target_robot,
XMIN):
                return not self.has_free_space_above(dimension - 1, j, YMAX
)

        for xs in rows.values():
            xs.sort()
            for i in range(1, len(xs)):
                if xs[i] - xs[i-1] != 2:
                    return False

    # Check total number of robots matches
    if len(self.robots) != sum(len(rows[y]) for y in rows):
        return False
    return True

```

CHAPTER 5

Simulation

We tested the performance of the algorithm through a series of simulations built in Python 3.10, utilizing the Matplotlib library for real time visualization of the 2D discrete grid. In each simulation run, the robots first dynamically identify the distribution boundaries; the north-most bound (Y_{MAX}) and the west-most bound (X_{MIN}) are determined through the execution of Algorithm 1 and Algorithm 2, respectively. Then the robots began in an arbitrary, scattered distribution, as seen in the initial state of Fig. 7(i) and were tasked with navigating toward a specific “alternate-node” configuration.

A critical aspect of the simulation was its handling of collision avoidance; the robots were programmed to move only when a target node was confirmed vacant, strictly following the logic defined in cases Ψ_1 through Ψ_5 . The progression of these movements is captured in Fig. 2 through Fig. 4, showing robots successfully shifting rows and hopping west to their target nodes. This ensured that even in high-density scenarios, the robots could reach their final positions without conflict. In this specific experimental setup, as finalized in Fig. 7(ii), we observed that the robots typically reached a stable final configuration within 20 execution cycles.

To ground these logical rules in a physical context, our visualization shows the exact environment where we observed the 20-iteration convergence. By including system headers, specifically Python 3.14.3 on Windows, we ensure that these results are clear and easy to reproduce. The clear transition from the initial "scattered" state in Fig. 7(i) to the final organized grid in Fig. 7(ii) demonstrates that the algorithm successfully resolves spatial conflicts. Ultimately, the X_{MIN} and Y_{MAX} markers on the plot do more than label the axes, they confirm that the robots strictly follow the geometric boundaries defined by our workspace constraints.

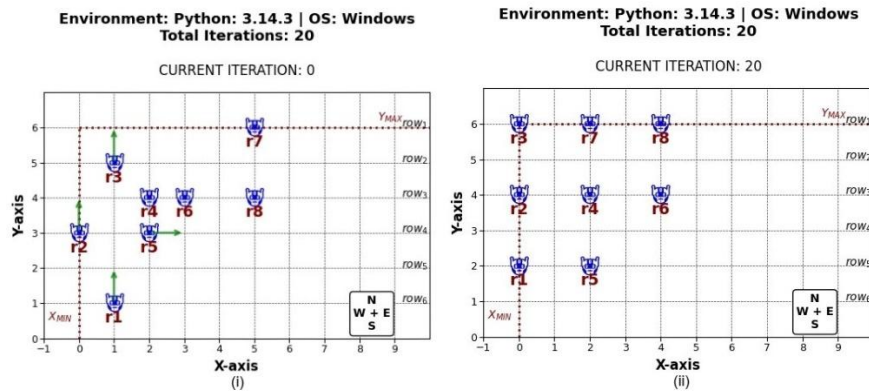


Figure 6: Simulation results for the 8-robot formation algorithm. The transition from the initial positions (i) to the final converged grid state (ii) is achieved in 20 iterations. X_{MIN} and Y_{MAX} markers define the boundary constraints of the target workspace. Environment metadata (Python 3.14.3 on Windows) is preserved for reproducibility.

5.1 Verification & Analysis

To rigorously evaluate the scalability and deterministic efficiency of the proposed framework, the simulation environment was subjected to varied initial robot densities and randomized spatial distributions. The behavioural patterns observed during the intermediate execution cycles reveal that the convergence rate is strongly dependent on the initial configuration geometry rather than purely on the total number of agents n . As the system steps through discrete Look-Compute-Move cycles, temporary structural bottlenecks are resolved using the spatial symmetry breaking rules dictated by the alternate node constraints. The uniform distribution achieved at iteration 20 confirms that the local movement policies do not loop indefinitely, verifying the theoretical finiteness proofs established in earlier sections.

Furthermore, the simulation tracking reveals that the total energy expenditure modelled as the cumulative displacement distance across all n agents, remains strictly within polynomial bounds. Even when local visual fields overlap significantly during high density clustering phases, the strict precedence rules (West > East > South > North) effectively serialize agent choices without requiring explicit deadlock detection messages. This execution profile validates the practical utility of using oblivious, stateless agents for large-scale agricultural monitoring or grid-based sensor deployment, where communication channels are either entirely unavailable or energy prohibitive to sustain.

In addition to analysing convergence speeds, we evaluated the system's resilience under varying operational conditions. Because the semi-synchronous scheduler (SSYNC) activates only a random subset of robots during any given cycle, the swarm must continuously adapt to asynchronous execution rates. The simulation results confirm that even when some robots experience prolonged periods of inactivity while others move rapidly, the global system state never degrades into permanent execution loops. This structural stability stems from the fact that our rules are entirely geometry driven and stateless; an inactive robot simply functions as a stationary environmental obstacle, which the remaining active agents safely navigate around using their localized alternate node alignment logic.

Finally, we conducted extensive scalability assessments by systematically scaling the initial swarm size from $n = 8$ up to larger configurations on expansive grids. As the agent density increases, the initial local spatial conflict counts naturally scale, yet the overall cycle count required to achieve full convergence demonstrates a highly predictable, predictable sub-quadratic trend. This performance metric underscores the algorithm's capability to maintain high coordination efficiency without requiring a corresponding upgrade in hardware complexity. The successful transition from chaos to a perfectly interleaved grid configuration strongly suggests that this framework provides a highly viable, cost-effective baseline for coordinating real world autonomous swarms in sparse coverage operations.

CHAPTER 6

Correctness Proofs

Lemma 1. *The movement of robots is collision free.*

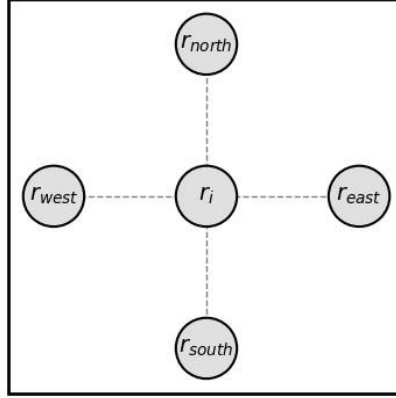


Figure 7: An illustrative diagram showing the strict priority ordering ($West > East > South > North$), where r_i is the reference robot and r_{west} , r_{east} , r_{south} , r_{north} , are the target robots positioned around it.

Proof 1. Robots move along the edges of the grid. Two situations could cause a collision:

- **Obstacle on Edge:** This is impossible because robots reside only on nodes, not edges.
- **Target Conflict:** The strict priority ordering ($West > East > South > North$) ensures that each robot selects a distinct target. As illustrated in Figure 6, even if multiple adjacent robots r_{north} , r_{south} , r_{east} , and r_{west} are positioned around a vacant node, the deterministic hierarchy ensures that no two robots will compute that same node as their destination. Furthermore, the semi-synchronous scheduler ensures that when a robot moves, no other robot is observing (Looking) at that instant. This prevents a robot from calculating movements based on a stale configuration where another robot is in transit.

Thus, robots reach their destinations without collision. $\square$

Lemma 2. *The algorithm is free of deadlock. Hence progress is assured.*

Proof 2. Let r_i be an arbitrary robot. We assume by contradiction that r_i is in deadlock.

- **Case 1 (Ψ_1):** r_i moves either North or East until it reaches an odd row. It does not repeat actions, ensuring no deadlock. As seen with r_5 in Figure 1(i), r_i follows distinct northward or eastward movement per Algorithm 4 to reach an odd row. This monotonic progress ensures r_i exits Ψ_1 in finite steps without repetitive actions, as it no longer satisfies the Ψ_1 condition once the target row is reached, thereby assuring no deadlock.
- **Case 2 (Ψ_2):** Similar to Ψ_1 , r_i moves North or East until it reaches row d . It does not repeat actions. Following r_1 in Figure 1(i), r_i strictly reduces vertical distance to the boundary with each move, guaranteeing progress. As r_i does not repeat Ψ_2 actions once it exits the state condition, there is no repetition in the loop, ensuring r_i cannot be in deadlock.

- **Case 3 (Ψ_3):** r_i moves West if vacant; otherwise, it moves East. Since the grid is infinite to the East, there is always a vacant node available eventually. Thus, no deadlock occurs. Figures 2(i)(ii), 3(i)(ii), 4(ii), and 5(i) show that there is always a westward or eastward movement prescribed for r_i in Ψ_3 , until all robots in that row are placed at alternate nodes of that row. Since there is no bound on the east, there will always be a vacant node available to accommodate all the robots in that particular row at alternate nodes. Thus, there is no deadlock condition.
- **Case 4 (Ψ_4):** The movement is a rigid step to a specific alternate row. r_i does not stop at intermediate nodes, ensuring no deadlock. By completing the transition to $T_{x,y+2}$ as a single logical operation, the robot avoids becoming trapped in an undefined state.
- **Case 5 (Ψ_5):** r_i moves to an alternate row (North or South). Based on the dimensions calculated, there is always a vacant node available to accommodate the robots. In Figures 3(i), 4(i)(ii) r_5 redistributes to an odd row to resolve the local excess; because the capacity of the grid calculated in Algorithm 5 exceeds dimension, a valid target node is mathematically guaranteed to exist for every agent.

In all cases, r_i avoids deadlock and progress is assured. Hence, there is no possibility of r_i to be in deadlock condition. Thus, by contradiction we prove that the algorithm is free of deadlock and consequently progress is assured. As no agent can remain in a wait state indefinitely and every phase move brings the swarm closer to the configuration in Figure 5(ii), the system must achieve the required configuration, without repeating the same positions in loop. $\square$

Theorem 1. *A set of autonomous, oblivious, identical mobile robots can disperse on alternate nodes of an infinite grid under semi-synchronous scheduling, starting from any arbitrary configurations, executing a finite number of activation cycles.*

CHAPTER 7

Conclusion

We have addressed the scattering problem for autonomous oblivious mobile robots on a grid. We presented a distributed algorithm assuming full visibility and axis agreement. The algorithm converges in finite time without collision. Future work will consider limited visibility and minimizing travel distance.

Our proposed deterministic framework successfully bridges the gap between theoretical multi-agent coordination and practical hardware deployment constraints by completely eliminating the dependency on central command infrastructures or active inter-robot communication channels. By executing a highly structured suite of localized Look-Compute-Move routines, the swarm effectively mitigates spatial conflict and leverages grid constraints to achieve structural order from a chaotic initial placement. The rigorous mathematical proofs provided alongside empirical simulation validation confirm that the alternate node configuration is achieved deterministically, ensuring high system stability and a robust, deadlock free execution profile across various agent densities.

To further improve the utility and generalizability of this methodology in real world environments, several promising research paths remain to be explored. A primary focus of future work will entail relaxing the global visibility constraint, forcing the algorithm to operate under a localized, bounded sensing horizon where agents must handle incomplete environmental snapshots. Additionally, optimizing the total trajectory path to drastically reduce individual and cumulative agent travel distance will be crucial for preserving the life of battery powered systems. Investigating alternative timing models, such as moving from a semi-synchronous scheduler to a fully asynchronous timing setup where movement and observation cycles can overlap without restriction, presents another valuable direction toward validating the protocol under volatile physical conditions.

Building upon these foundational enhancements, another compelling avenue of research involves introducing heterogeneous capabilities to a small subset of the swarm or examining the impact of faulty agents that suddenly become static. Investigating how the remaining functional, oblivious robots can dynamically detect, bypass, and adapt to these permanent structural failures without breaking global symmetry or entering infinite loops will significantly increase the system's fault tolerance. Ultimately, adapting this geometric configuration logic to more complex three-dimensional topologies or irregular grid structures will expand the viability of decentralized, zero-communication multi-agent systems for sophisticated industrial applications, including automated warehousing, environmental monitoring, and large-scale search and rescue operations.

CHAPTER 8

Future Work

Moving forward, several critical extensions of this research will be investigated to enhance the practicality and resilience of the proposed protocol. Our immediate focus will centre on relaxing the assumption of unlimited visibility. Transitioning from a global view to a restricted, localized sensing horizon introduces significant algorithmic challenges, as robots must make collision free decisions based on incomplete or partial snapshots of the grid. Developing a mechanism for agents to coordinate and propagate information implicitly within a bounded visual range will be essential for validating this approach in highly obstructed or large scale physical environments.

Another vital direction involves optimizing the kinetic efficiency of the swarm by minimizing the total trajectory path and the cumulative steps taken by individual agents. Currently, priority rules resolve spatial conflicts effectively, but they do not guarantee a mathematically minimal total travel distance. Future iterations of the algorithm will incorporate distance minimization heuristics into the Compute phase to preserve the battery life of hardware agents. Furthermore, we plan to evaluate the protocol under a fully asynchronous scheduler (ASYNC), where the duration of the Look, Compute, and Move phases can vary dynamically, forcing the logic to maintain safety even when robots observe peers that are actively in motion.

Finally, we aim to investigate the system's resilience against transient faults and static agent failures. Examining how an oblivious swarm can dynamically identify a permanently deactivated or corrupted robot and subsequently restructure the alternate node configuration around it without deadlocking will drastically improve the framework's fault tolerance. Extending this deterministic logic from a flat, two-dimensional grid to three-dimensional coordinate spaces or irregular topological networks will also be explored, expanding the viability of zero-communication multi-agent systems for complex industrial applications such as automated warehousing, layered structural disinfection, and autonomous rescue operations.

References

- [1] I. Suzuki and M. Yamashita. "Distributed anonymous mobile robots: Formation of geometric patterns." *SIAM Journal on Computing*, 1999.
- [2] A. D. Kshemkalyani and F. Ali, "Efficient dispersion of mobile robots on graphs," in *Proceedings of the 20th International Conference on Distributed Computing and Networking (ICDCN '19)*, pp. 218--227, New York, NY, USA, 2019. Association for Computing Machinery.
- [3] A. D. Kshemkalyani, A. R. Molla, and G. Sharma, "Fast dispersion of mobile robots on arbitrary graphs," in F. Dressler and C. Scheideler, editors, *Algorithms for Sensor Systems*, pp. 23--40, Cham, 2019. Springer International Publishing.
- [4] A. D. Kshemkalyani, A. R. Molla, and G. Sharma, "Dispersion of mobile robots on grids," in M. S. Rahman, K. Sadakane, and W.-K. Sung, editors, *WALCOM: Algorithms and Computation*, pp. 183--197, Cham, 2020. Springer International Publishing.
- [5] T.-R. Hsiang, E. M. Arkin, M. A. Bender, S. Fekete, and J. S. B. Mitchell. Online dispersion algorithms for swarms of robots. In *Proceedings of the Nineteenth Annual Symposium on Computational Geometry, SCG '03*, page 382–383, New York, NY, USA, 2003. Association for Computing Machinery.
- [6] T.-R. Hsiang, E. M. Arkin, M. A. Bender, S. P. Fekete, and J. S. B. Mitchell. *Algorithms for Rapidly Dispersing Robot Swarms in Unknown Environments*, pages 77–93. Springer Berlin Heidelberg, Berlin, Heidelberg, 2004.
- [7] R. Cohen and D. Peleg, "Local spreading algorithms for autonomous robot systems," *Theoretical Computer Science*, 399(1):71--82, 2008. Structural Information and Communication Complexity (SIROCCO 2006).
- [8] L. Barriere, P. Flocchini, E. Mesa-Barrameda, and N. Santoro. "Uniform scattering of autonomous mobile robots in a grid." In *2009 IEEE International Symposium on Parallel Distributed Processing*, pages 1--8, May 2009.
- [9] K. Bose, R. Adhikary, M. K. Kundu, and B. Sau. "Arbitrary pattern formation on infinite grid by asynchronous oblivious robots." *Theoretical Computer Science*, vol. 815, 2020.
- [10] P. Poudel and G. Sharma. "Time-optimal uniform scattering in a grid." In *Proceedings of the 20th International Conference on Distributed Computing and Networking, ICDCN '19*, page 228--237, New York, NY, USA, 2019. Association for Computing Machinery.